Synchronization

Introduction to Synchronization

Stefano Quer
Dipartimento di Automatica e Informatica
Politecnico di Torino

License Information

This work is licensed under the license

CC BY-NC-ND 4.0

Attribution-NonCommercial-NoDerivatives 4.0 International

This license requires that reusers give credit to the creator. It allows reusers to copy and distribute the material in any medium or format in unadapted form and for noncommercial purposes only.

BY: Credit must be given to you, the creator.

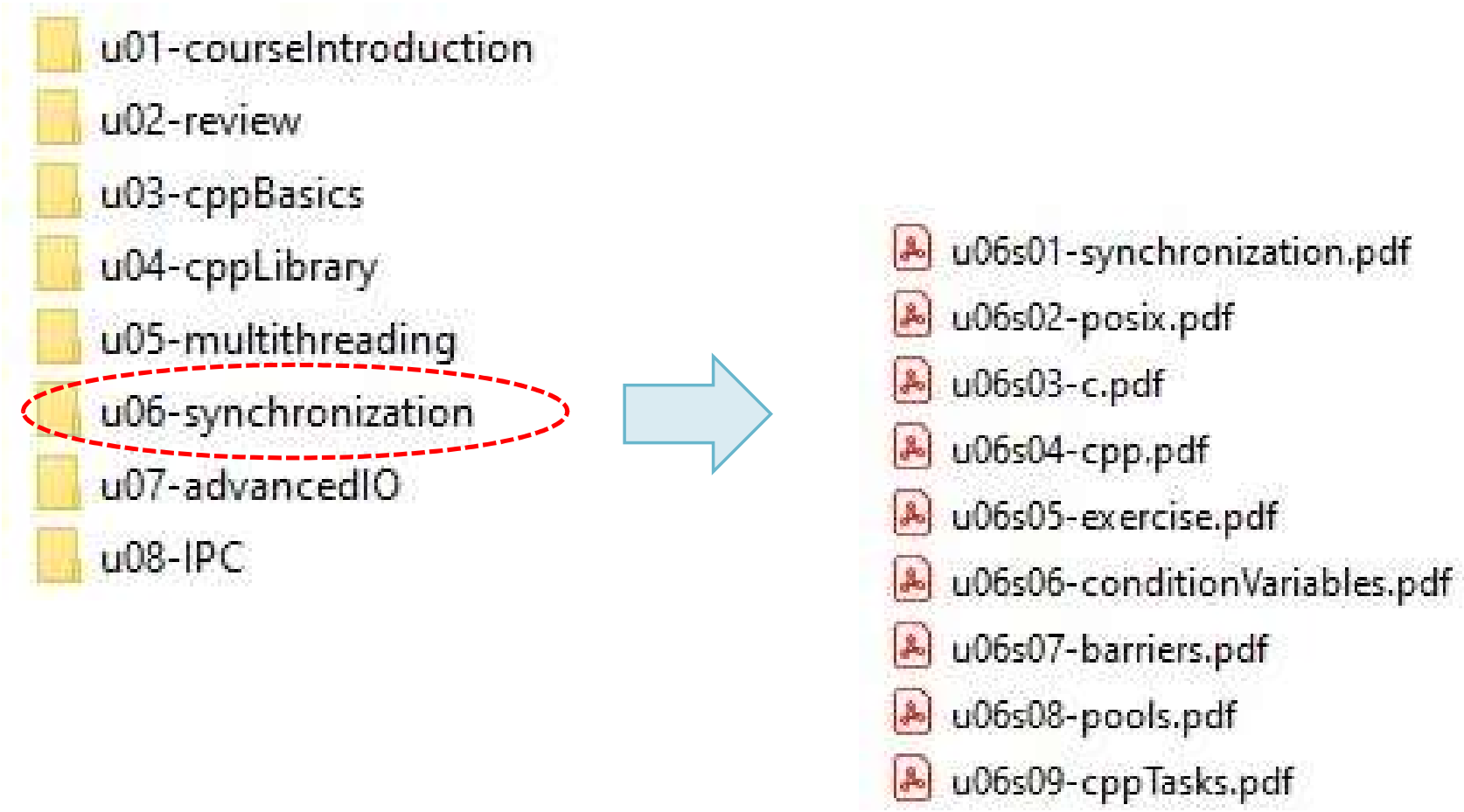
NC: Only noncommercial use of your work is permitted.
Noncommercial means not primarily intended for or directed towards commercial advantage or monetary compensation.

ND: No derivatives or adaptations of your work are permitted.

To view a copy of the license, visit:
https://creativecommons.org/licenses/by-nc-nd/4.0/?ref=chooser-v1

Introduction

- Where are we?



Introduction

- Critical Section (**CS**) or Critical Region (**CR**)
  - A section of code, common to multiple threads, in which each thread can read and **write** shared objects
- Access to CS is subject to **race conditions**
  - The result depends on the execution order of the processes instructions

Example

FIFO, Queue, Circular Buffer

T_i

```
void enqueue (int val) {
  if (n>SIZE) return;
  queue[tail] = val;
  tail=(tail+1)%SIZE;
  n++;
  return;
}
```

register = n
register = register + 1
n = register

T_j

```
int dequeue (int *val) {
  if (n<=0) return;
  *val=queue[head];
  head=(head+1)%SIZE;
  n--;
  return;
}
```

register = n
register = register - 1
n = register

- Even if **enqueue** and **dequeue** operate on the different ends of the queue, the variable **n** is shared

Race condition:
Increments and decrements can be lost

Critical sections

- Race conditions could be prevented with an access protocol that enforces mutual exclusion for each CS
  - Before a CS, there should be a **reservation** section
    - The reservation code must block (lock out) the P (or T) if another P (or T) is using its CS
  - After the CS, there should be a **release** section
    - The release possibly unlocks another P (or T) which was waiting in the ”reservation” code of its CS

Access protocol

T_i

```
while (TRUE) {
   ...
   reservation code
   Critical Section
   release code
   ...
   non critical section
}
```

T_j

```
while (TRUE) {
   ...
   reservation code
   Critical Section
   release code
   ...
   non critical section
}
```

- Every Critical Section is protected by an
  - Enter code (reservation, or prologue)
  - Exit code (release, or epilogue)
- Non-critical sections should not be protected
- OSs provide appropriate primitives